

Complete SQL from Basics to Advanced for Placements | All SQL Queries | SQL for Analyst

16 min read · Oct 21, 2025



Sheel Ganvir



In this Medium article, I've compiled all the important SQL concepts and queries you need from basics to advanced specifically tailored for placement preparation.

After going through multiple trusted resources, I've gathered everything in one comprehensive guide to help you master SQL for interviews, data analysis, and real-world problem-solving.

COMPLETE SQL

from Basics to Advanced



FOR PLACEMENTS

All SQL Queries SQL for Analyst



Content of this blog

1. What is SQL?

- Object
- Class

2. SQL DDL Commands

- CREATE
- ALTER
- DROP
- TRUNCATE
- RENAME

3. SQL DML Commands

- INSERT
- SELECT
- UPDATE
- DELETE

4. ORDER BY (ASC/DESC)

5. GROUP BY

6. SQL JOINS

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL OUTER JOIN
- CROSS JOIN
- SELF JOIN

7. Keys and Constraints

- Primary Key
- Foreign Key
- CHECK
- UNIQUE
- NOT NULL
- DEFAULT

1. What is SQL?

- SQL = Structured Query Language.
- It's how you ask questions (queries) to a **database** (structured storage of tables, rows, columns).
- Think: Excel sheets → SQL database tables (but with rules, security, concurrency, types)

Why use SQL vs Excel?

- Handles concurrent changes safely (no accidental overwrites).
- Enforces **data types** and constraints (prevents wrong values).
- Scales to millions of rows and supports complex queries (joins, aggregation).

2. Database basics — quick terms

- **Database:** container for tables (like a workbook).
- **Table:** collection of rows (like a sheet).

- Row (record) = one entry.
- Column = attribute/field (has a data type).
- Query = question you ask the DB (SELECT, INSERT, UPDATE...). ?

3. Main SQL categories (very important)

- DDL (Data Definition Language): CREATE, ALTER, DROP — define structure.
- DML (Data Manipulation Language): SELECT, INSERT, UPDATE, DELETE — operate on data.
- DCL (Data Control Language): GRANT, REVOKE .
- TCL (Transaction Control Language): BEGIN/START TRANSACTION, COMMIT, ROLLBACK .

4. Data types (common ones you actually use)

- INT — integers (e.g., roll_no).
- BIGINT — very large integers (credit card IDs, huge counters).
- FLOAT / DECIMAL — decimal numbers (money, metrics).
- CHAR(n) / VARCHAR(n) — text (fixed/variable length).
- BOOLEAN — true/false.
- DATE, DATETIME, TIMESTAMP — dates and times.

5. Create database & tables — examples

Create database:

```
CREATE DATABASE test_one;
USE test_one; -- select the database
```

Create table `students`:

```
CREATE TABLE students (  
  roll_no INT,          -- small range: e.g., up to 99  
  name VARCHAR(100),  
  grade CHAR(1)         -- 'A','B','C' etc  
);
```

Insert rows:

```
INSERT INTO students (roll_no, name, grade)  
VALUES  
  (1, 'Virat', 'A'),  
  (2, 'Rohit', 'B');
```

Select:

```
SELECT * FROM students;
```

roll_no	name	grade
1	Virat	A
2	Rohit,	B

. . .

SQL DDL Commands

What is DDL?

DDL = Data Definition Language

DDL commands are used to **define or modify** the structure of the database like tables, columns, or entire databases.

There are 5 Commands under DDL:

1. CREATE
2. ALTER
3. DROP
4. TRUNCATE
5. RENAME

Let's understand each one — with examples and real talk 😎

1. CREATE Command

Use: To create a new database or a table.

Syntax:

```
CREATE DATABASE Class;
USE Class;

CREATE TABLE Student_Data (
    RollNo INT,
    Name VARCHAR(50),
    Marks DECIMAL(5,2),
    Grade CHAR(1)
);
```

- CREATE DATABASE → makes a new database.
- USE → selects that database.
- CREATE TABLE → defines a new table with columns and data types.

Insert data example:

```
INSERT INTO Student_Data (RollNo, Name, Marks, Grade)
VALUES
(1, 'Arjun', 95.50, 'A'),
(2, 'Anupam', 88.75, 'B'),
(3, 'Ishan', 78.25, 'C'),
```

```
(4, 'Krish', 92.00, 'A'),  
(5, 'Moksh', 85.50, 'B'),  
(6, 'Nitin', 77.00, 'C'),  
(7, 'Parin', 90.25, 'A'),  
(8, 'Rishi', 83.75, 'B'),  
(9, 'Rohit', 72.50, 'C'),  
(10, 'Rahul', 94.75, 'A');
```

Check data:

```
SELECT * FROM Student_Data;
```

STUDENT_DATA

RollNo	Name	Marks	Grade
1	Arjun	95.50	A
2	Anupam	88.75	B
3	Ishan	78.25	C
4	Krish	92.00	A
5	Moksh	85.50	B
6	Nitin	77.00	C
7	Parin	83.75	A
8	Rishi	83.75	A
9	Rahul	94.75	A

2. ALTER Command (Change karne ke liye)

ALTER means to change or modify.

When we need to add, **modify**, or **delete** a **column** in an existing table — we use ALTER.

(a) Add a new column

Example:

```
ALTER TABLE Student_Data  
ADD COLUMN Result CHAR(10);
```

✓ Adds a new column `Result` to the table.

✿ After Adding the Column (`ADD COLUMN Result CHAR(10);`)

Field	Type	Null	Key	Default	Extra	
RollNo	INT	YES		NULL		
Name	VARCHAR(50)	YES		NULL		
Marks	DECIMAL(5,2)	YES		NULL		
Grade	CHAR(1)	YES		NULL		
Result	CHAR(10)	YES		NULL		

(b) Modify an existing column

Example:

```
ALTER TABLE Student_Data  
MODIFY COLUMN Result INT(10);
```

This command changes the data type of **Result** from **CHAR** to **INT**.

Check structure:

```
DESC Student_Data;
```


You'll see:

 After Modifying the Column (`MODIFY COLUMN Result INT(10);`)

Field	Type	Null	Key	Default	Extra
RollNo	INT	YES		NULL	
Name	VARCHAR(50)	YES		NULL	
Marks	DECIMAL(5,2)	YES		NULL	
Grade	CHAR(1)	YES		NULL	
Result	INT(10)	YES		NULL	

 After Adding the Column (`ADD COLUMN Result CHAR(10);`)

Field	Type	Null	Key	Default	Extra	
RollNo	INT	YES		NULL		
Name	VARCHAR(50)	YES		NULL		
Marks	DECIMAL(5,2)	YES		NULL		
Grade	CHAR(1)	YES		NULL		
Result	CHAR(10)	YES		NULL		

Change it back

```
ALTER TABLE Student_Data
MODIFY COLUMN Result CHAR(10);
```

After Adding the Column (`ADD COLUMN Result CHAR(10);`)

Field	Type	Null	Key	Default	Extra	
RollNo	INT	YES		NULL		
Name	VARCHAR(50)	YES		NULL		
Marks	DECIMAL(5,2)	YES		NULL		
Grade	CHAR(1)	YES		NULL		
Result	CHAR(10)	YES		NULL		

(d) Drop a column

Example:

```
ALTER TABLE Student_Data  
DROP COLUMN Result;
```

🔧 This completely removes the `Result` column from your table.

Table Structure After Dropping `Result` Column

Field	Type	Null	Key	Default	Extra
RollNo	INT	YES		NULL	
Name	VARCHAR(50)	YES		NULL	
Marks	DECIMAL(5,2)	YES		NULL	
Grade	CHAR(1)	YES		NULL	

STUDENT_DATA

RollNo	Name	Marks	Grade
1	Arjun	95.50	A
2	Anupam	88.75	B
3	Ishan	78.25	C
4	Krish	92.00	A
5	Moksh	85.50	B
6	Nitin	77.00	C
7	Parin	83.75	A
8	Rishi	83.75	A
9	Rahul	94.75	A

3. DROP Command

Used to **delete** the whole **table** (not just data, but structure also).

Syntax:

```
DROP TABLE Student_Data;
```



This will remove the entire table permanently — data + structure both gone!

Check with:

```
SHOW TABLES;
```

Table will no longer exist.

4. TRUNCATE Command

TRUNCATE is like **delete all data** from table — but table structure stays.

Example:

```
TRUNCATE TABLE Student_Data;
```

✓ Removes all rows from table, but table still exists (you can still insert new data later).

Check:

```
SELECT * FROM Student_Data;
```

It'll show empty result — but columns are still there.

RollNo	Name	Marks	Grade
(no rows)			

5. RENAME Command

Used to **change table name**.

Syntax:

```
ALTER TABLE Student_Data  
RENAME TO Students_New;
```

Now your table name is changed to `Students_New`.

Check:

```
SHOW TABLES;
```

Interview / Viva Tips

DDL vs DML

- DDL → defines structure (CREATE, ALTER, DROP, TRUNCATE, RENAME)
- DML → manipulates data (INSERT, UPDATE, DELETE)

 DDL commands are **auto-commit** → once executed, you can't roll them back easily.

 Always double-check before `DROP` or `TRUNCATE` 😄

 Remember ALTER has 3 powers → **ADD, MODIFY, DROP** column.

. . .

What is DML?

DML = Data Manipulation Language

 “Manipulation” means modifying or working with existing data inside the database — like **inserting, reading, updating, or deleting** rows.

These are the most frequently used SQL commands in any project or company.

DML Commands List

Step 1: Create Database & Table

```
CREATE DATABASE DML;  
USE DML;  
  
CREATE TABLE Students (  
    Student_id INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Age INT,
```

```
Grade VARCHAR(2)
);
```

This creates a table `Students` with columns — `Student_id`, `Name`, `Age`, and `Grade`.

Step 2: INSERT Command

Used to add new rows of data into the table.

Syntax:

```
INSERT INTO Students (student_id, name, age, grade)
VALUES (1, 'John Doe', 18, 'A');
```

✓ You can also insert multiple rows together:

```
INSERT INTO Students (student_id, name, age, grade)
VALUES
(2, 'Jane Smith', 19, 'B'),
(3, 'Bob Johnson', 20, 'C'),
(4, 'Alice Brown', 18, 'A'),
(5, 'Eva White', 21, 'A'),
(6, 'David Lee', 22, 'B');
```

Table view after inserting:

Table: Students			
Student_id	Name	Age	Grade
1	John Doe	18	A
2	Jane Smith	19	B
3	Bob Johnson	20	C
4	Alice Brown	18	A
5	Eva White	21	A
6	David Lee	22	B

✓ Tip: The order of columns in the `VALUES` section must match the order you define in parentheses after `INSERT INTO`.

Step 3: SELECT Command

Used to view data from the table.

Syntax:

```
SELECT column_name FROM table_name;
```

Example:

```
SELECT Name FROM Students;
```

Output:

Name
John Doe
Jane Smith
Bob Johnson
Alice Brown
Eva White
David Lee

If you want to see **multiple columns**, just list them:

```
SELECT Name, Grade FROM Students;
```

If you want **all columns**, use `*`:

```
SELECT * FROM Students;
```



"*" means "select all columns".

Step 4: UPDATE Command

Used to modify an existing record in the table.

Syntax:


```
UPDATE table_name  
SET column_name = new_value  
WHERE condition;
```

Example:

```
UPDATE Students  
SET Name = 'John Cena'  
WHERE student_id = 1;
```

➡ Here, we are changing the name of the student with `student_id = 1`.

After Update:

Student_id	Name	Age	Grade
1	John Cena	18	A
2	Jane Smith	19	B
3	Bob Johnson	20	C
4	Alice Brown	18	A
5	Eva White	21	A
6	David Lee	22	B

✅ Always use the `WHERE` condition in `UPDATE`, or all rows will get updated — which is dangerous!

❌ Step 5: DELETE Command

Used to remove data from the table.

Syntax:

```
DELETE FROM table_name  
WHERE condition;
```

Example:

```
DELETE FROM Students  
WHERE student_id = 101;
```

This will delete the student whose `student_id = 101`.

✅ Again, remember:

If you forget the `WHERE` clause → it deletes **all rows** 😱

▼ ORDER BY Clause — Sorting Results

Now that you know how to insert, view, update, and delete data...

Let's learn how to **arrange** that data in a meaningful order using `ORDER BY`.

What is ORDER BY?

The `ORDER BY` clause sorts the result of a query in either **ascending (ASC)** or **descending (DESC)** order.

Default is **ASCENDING** if not mentioned.

Example 1: Sort by Age (Ascending)

```
SELECT * FROM Students  
ORDER BY Age;
```

Output:

Student_id	Name	Age	Grade	
1	John Cena	18	A	
4	Alice Brown	18	A	
2	Jane Smith	19	B	
3	Bob Johnson	20	C	
5	Eva White	21	A	
6	David Lee	22	B	

Example 2: Sort by Age (Descending)

```
SELECT * FROM Students
ORDER BY Age DESC;
```

Now it shows highest age first:

Student_id	Name	Age	Grade
6	David Lee	22	B
5	Eva White	21	A
3	Bob Johnson	20	C
2	Jane Smith	19	B
1	John Cena	18	A
4	Alice Brown	18	A

Example 3: Sort by Multiple Columns

👉 Sort by **Grade (Ascending)** and then by **Age (Descending)**

```
SELECT * FROM Students
```

```
ORDER BY Grade ASC, Age DESC;
```

- First sorts students by Grade (A → B → C)
- Then, for same Grade, sorts by Age (highest age first).

Output:

Student_id	Name	Age	Grade	
5	Eva White	21	A	
1	John Cena	18	A	
4	Alice Brown	18	A	
6	David Lee	22	B	
2	Jane Smith	19	B	
3	Bob Johnson	20	C	

Example 4: Sort by Name (Alphabetical Order)

```
SELECT * FROM Students  
ORDER BY Name;
```

Output:

Student_id	Name	Age	Grade
4	Alice Brown	18	A
3	Bob Johnson	20	C
6	David Lee	22	B
5	Eva White	21	A
2	Jane Smith	19	B
1	John Cena	18	A

✅ This is also called **Alphabetical Ordering**.

📖 Combining Orders

You can mix ascending and descending:

```
SELECT * FROM Students
ORDER BY Grade ASC, Age DESC;
```

👉 This means: within each Grade (A, B, C), show higher ages first.

Student_id	Name	Age	Grade
5	Eva White	21	A
1	John Cena	18	A
4	Alice Brown	18	A
6	David Lee	22	B
2	Jane Smith	19	B
3	Bob Johnson	20	C

• • •

What is GROUP BY?

The `GROUP BY` clause in SQL is used to **group rows that have the same values** in one or more columns —
then you can apply **aggregate functions** (like `SUM`, `COUNT`, etc.) to each group.

You can think of it like this 🙌

“If I have hundreds of sales transactions, `GROUP BY` helps me find totals or averages for each category — like per customer, per payment method, per city, etc.”

Real-Life Example — “Green Plant Store”

Let’s imagine I work at a company called **Green Plant Store** 🌿 —
It’s an online platform in Bangalore that sells plants for home delivery.

My manager gives me a task:

“Find out which **payment method** was used **most frequently** this week, and also which one brought in the **highest total amount**.”

So we’ll solve this using SQL with the help of `GROUP BY`.

Step 1: Create the Table

Here’s the SQL command to create our table:

```
CREATE TABLE Orders (  
  Customer_ID VARCHAR(5),  
  Payment_Mode VARCHAR(20),  
  Amount INT(20)  
);
```

✅ Columns Explanation:

- `Customer_ID` → Unique ID for each customer
- `Payment_Mode` → How they paid (like UPI, Credit Card, Debit Card, etc.)
- `Amount` → How much they paid

Step 2: Insert Data

Let's insert a few sample rows of data —
(similar to what you'd see in a weekly order report)

```
INSERT INTO Orders (Customer_ID, Payment_Mode, Amount)
VALUES
('C001', 'Credit Card', 1010),
('C002', 'UPI', 980),
('C003', 'Credit Card', 670),
('C004', 'UPI', 1276),
('C005', 'Debit Card', 670),
('C006', 'Credit Card', 750),
('C007', 'UPI', 2400),
('C008', 'Credit Card', 1010),
('C009', 'Debit Card', 500),
('C010', 'Credit Card', 2000);
```

✅ Now we've got 10 orders — each with a customer, payment mode, and amount.

Step 3: View the Table

Before grouping anything, let's check our data:

```
SELECT * FROM Orders;
```

Output:

Customer_ID	Payment_Mode	Amount	
C001	Credit Card	1010	
C002	UPI	980	
C003	Credit Card	670	
C004	UPI	1276	
C005	Debit Card	670	
C006	Credit Card	750	
C007	UPI	2400	
C008	Credit Card	1010	
C009	Debit Card	500	
C010	Credit Card	2000	

Step 4: Use GROUP BY with SUM()

Now, we want to find the **total amount collected per payment method**.

So we'll use `SUM()` along with `GROUP BY`.

```
SELECT
  Payment_Mode,
  SUM(Amount) AS Total_Amount
FROM Orders
GROUP BY Payment_Mode;
```

Output:

💰 Result of `GROUP BY Payment_Mode`

Payment_Mode	Total_Amount
Credit Card	6440
Debit Card	1170
UPI	4656

✅ This shows:

- Total received through Credit Card = ₹5440
- Total received through UPI = ₹4656
- Total received through Debit Card = ₹1170

Now we know **Credit Card** brought in the **highest total amount** this week.

Step 5: Use **GROUP BY** with **COUNT()**

Now let's find how many times each payment mode was used (number of transactions).

```
SELECT
  Payment_Mode,
  COUNT(Amount) AS Mode_Counts
FROM Orders
GROUP BY Payment_Mode;
```

Output:

Payment_Mode	Mode_Counts
Credit Card	5
Debit Card	2
UPI	3

✓ This means:

- **Credit Card** was used **5 times**
- **UPI** was used **3 times**
- **Debit Card** was used **2 times**

So, depending on what your manager asked —
you can answer in two ways:

- “Credit Card” was **used most often** (5 transactions)
- “Credit Card” also gave the **highest revenue** (₹5440)

Understanding GROUP BY Conceptually

Imagine Excel:

You have a column for Payment Mode and another for Amount.

If you want the total amount per mode —
you’ll apply a **SUMIF()** formula for each payment mode.
That’s exactly what SQL’s **GROUP BY** does automatically.

Step 6: Combine with ORDER BY (Optional)

Now, let’s sort the result by total amount — highest to lowest.

```
SELECT
  Payment_Mode,
  SUM(Amount) AS Total_Amount
FROM Orders
GROUP BY Payment_Mode
ORDER BY Total_Amount DESC;
```

Output:

Payment_Mode	Total_Amount
Credit Card	5440
UPI	4656
Debit Card	1170

✅ This helps you instantly identify the **most used** or **most profitable** payment method.

Interview Tip

Q: What is the difference between `WHERE` and `HAVING` in GROUP BY?

A:

- `WHERE` filters **before** grouping
- `HAVING` filters **after** grouping

Example:

```
SELECT Payment_Mode, SUM(Amount)
FROM Orders
GROUP BY Payment_Mode
HAVING SUM(Amount) > 2000;
```

→ This shows only payment modes whose total exceeds ₹2000.

Quick Recap

- ✅ GROUP BY is used with **aggregate functions** to summarize data
- ✅ It combines rows with same column values into one group
- ✅ You can pair it with **ORDER BY** for better readability
- ✅ Real-world usage: sales reports, payment analysis, customer grouping, etc.

• • •

SQL JOINS

Yes, the *heart and soul* of SQL.

I can confidently tell you — if SQL were a cricket team, then **JOIN** would be its *captain* 🏏

Because without JOINS, you simply can't do real-world analysis in SQL.

Whenever your manager or interviewer gives you a real-world problem — you'll almost always need to use **JOINS** to combine data from multiple tables.

Why Do We Need JOINS?

In Excel, you sometimes have data in **two separate sheets** — say one sheet has employee names and IDs, and another has their salaries or departments.

To combine both, you use **VLOOKUP** or **XLOOKUP** — right?

In SQL, instead of lookup formulas, we use **JOINS**.

Because JOINS help us connect multiple tables that share **common columns** (like IDs or keys).

That's the beauty of relational databases — data is **spread across multiple tables**, but **linked logically** using keys.

The Tables We'll Use

Table	Description
employees	Stores employee details
departments	Contains department IDs and names
salaries	Holds employee salary info
projects	Lists projects and project leads

Each table is connected to others using **common keys** (like `emp_id`, `department_id`, etc.)

So, this is how we'll visualize it:

- └─ Departments
- └─ Employees
- └─ Salaries
- └─ Projects

Each employee belongs to a department, has a salary, and might be assigned to a project.

SQL Tables Setup:

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(100),  
    emp_department_id INT  
);  
CREATE TABLE departments (  
    department_id INT PRIMARY KEY,  
    department_name VARCHAR(100)  
);  
CREATE TABLE salaries (  
    emp_id INT PRIMARY KEY,  
    salary_amount DECIMAL(10,2),  
    FOREIGN KEY (emp_id) REFERENCES employees(emp_id)  
);  
CREATE TABLE projects (  
    project_id INT PRIMARY KEY,  
    project_name VARCHAR(100),  
    project_lead_id INT,  
    FOREIGN KEY (project_lead_id) REFERENCES employees(emp_id)  
);
```

Departments		Projects			Employees			Salaries	
department_id	department_name	project_id	project_name	project_lead_id	emp_id	emp_name	emp_department_id	emp_id	salary_amount
101	HR	1	Project A	1	1	JohnDoe	101	1	60000
102	IT	2	Project B	2	2	JaneSmith	102	2	75000
103	Marketing	3	Project C	3	3	BobJohnson	101	3	55000
104	Finance	4	Project D	4	4	AliceBrown	102	4	80000
105	Operations	5	Project E	5	5	CharlieDavis	101	5	65000
					6	EvaWhite	103	6	70000
					7	FrankJones	103	7	60000
					8	GraceMiller	101	8	85000
					9	HarryWilson	102	9	75000
					10	IvyMartin	103	10	90000
					11	JackAnderson	102	11	70000
					12	KellyThomas	101	12	60000
					13	LeoHarris	103	13	80000
					14	MiaGarcia	101	14	55000
					15	NickTaylor	102	15	75000
					16	OliviaClark	103	16	70000
					17	PaulaAllen	102	17	85000
					18	QuinnScott	101	18	60000
					19	RandyWard	103	19	80000
					20	SarahHall	102	20	70000
					21	Mart Ward	109		
					22	Sam Hall	107		

✓ These represent a real company's structure.

What Are JOINS?

In simple words:

A **JOIN** is used to combine rows from two or more tables based on a related column between them.

We'll learn **6 types of JOINS** today:

1. INNER JOIN
2. LEFT JOIN
3. RIGHT JOIN
4. FULL OUTER JOIN
5. CROSS JOIN
6. SELF JOIN

(We'll cover SELF JOIN briefly at the end, since it's slightly special.)

LET'S BEGIN — SQL JOINS ONE BY ONE

1. INNER JOIN — “Show only matching data”

Problem Statement:

Retrieve a list of employees along with their department information — show only those employees who belong to an existing department.

Query:

```
SELECT
    e.emp_id,
    e.emp_name,
    d.department_name
FROM employees e
INNER JOIN departments d
ON e.emp_department_id = d.department_id;
```

✓ Explanation:

- `INNER JOIN` returns **only matching records** from both tables.
- If an employee's `emp_department_id` exists in the `departments` table, it will appear in the result.
- If not, that employee will be *excluded*.

Output (example):

emp_id	emp_name	department_name
1	JohnDoe	HR
2	JaneSmith	IT
3	BobJohnson	HR
4	AliceBrown	IT
5	CharlieDavis	HR
6	EvaWhite	Marketing
7	FrankJones	Marketing
8	GraceMiller	HR
9	HarryWilson	IT
10	IvyMartin	Marketing
11	JackAnderson	IT
12	KellyThomas	HR
13	LeoHarris	Marketing
14	MiaGarcia	HR
15	NickTaylor	IT
16	OliviaClark	Marketing
17	PaulaAllen	IT
18	QuinnScott	HR
19	RandyWard	Marketing
20	SaraHall	IT

 Use *INNER JOIN* when you only want records that exist in both tables.

2. LEFT JOIN — “Include everyone from the left table”

Problem Statement:

Display a list of all employees and their salary information — including employees who do **not** have a salary entry yet.

Query:

```
SELECT
  e.emp_id,
```



```
e.emp_name,  
s.salary_amount  
FROM employees e  
LEFT JOIN salaries s  
ON e.emp_id = s.emp_id;
```

✓ Explanation:

- LEFT JOIN returns all rows from the left table (`employees`)
- If no match is found in the right table (`salaries`), it fills NULL values.
- So even if a new employee doesn't have a salary entry yet, they'll still appear.

Output (example):

emp_id	emp_name	salary_amount	
1	JohnDoe	60000	
2	JaneSmith	75000	
3	BobJohnson	55000	
4	AliceBrown	80000	
5	CharlieDavis	65000	
6	EvaWhite	70000	
7	FrankJones	60000	
8	GraceMiller	85000	
9	HarryWilson	75000	
10	IvyMartin	90000	
11	JackAnderson	70000	
12	KellyThomas	60000	
13	LeoHarris	80000	
14	MiaGarcia	55000	
15	NickTaylor	75000	
16	OliviaClark	70000	
17	PaulaAllen	85000	
18	QuinnScott	60000	
19	RandyWard	80000	
20	SaraHall	70000	
21	Mart Ward	NULL	
22	Sam Hall	NULL	

 Use *LEFT JOIN* when you want all records from the main (left) table, even if some have missing details.

3. RIGHT JOIN — “Include everyone from the right table”

Problem Statement:

Generate a report of all salary records along with employee info — ensuring all salary entries are included (even if employee info is missing).

Query:

```
SELECT
  e.emp_id,
  e.emp_name,
  s.salary_amount
FROM salaries s
RIGHT JOIN employees e
ON e.emp_id = s.emp_id;
```

	emp_id	emp_name	salary_amount	
	1	JohnDoe	60000	
	2	JaneSmith	75000	
	3	BobJohnson	55000	
	4	AliceBrown	80000	
	5	CharlieDavis	65000	
	6	EvaWhite	70000	
	7	FrankJones	60000	
	8	GraceMiller	85000	
	9	HarryWilson	75000	
	10	IvyMartin	90000	
	11	JackAnderson	70000	
	12	KellyThomas	60000	
	13	LeoHarris	80000	
	14	MiaGarcia	55000	
	15	NickTaylor	75000	
	16	OliviaClark	70000	
	17	PaulaAllen	85000	
	18	QuinnScott	60000	
	19	RandyWard	80000	
	20	SaraHall	70000	
	21	Mart Ward	NULL	
	22	Sam Hall	NULL	

✓ Explanation:

- RIGHT JOIN does the **opposite of LEFT JOIN**.
- It includes **all rows from the right table** (`employees`), and fills NULLs where there's no match in the left (`salaries`).

🎯 Think of it as flipping LEFT JOIN horizontally.

4. FULL OUTER JOIN — “Include everything from both sides”

Problem Statement:

Create a comprehensive report including both employees and project information —
show all employees and all projects, even if they're not currently linked.

Query:

```
SELECT
  e.emp_id,
  e.emp_name,
  p.project_name
FROM employees e
FULL OUTER JOIN projects p
ON e.emp_id = p.project_lead_id;
```

 Result Table

emp_id	emp_name	project_name	
1	JohnDoe	Project A	
2	JaneSmith	Project B	
3	BobJohnson	Project C	
4	AliceBrown	Project D	
5	CharlieDavis	Project E	
6	EvaWhite	NULL	
7	FrankJones	NULL	
...	...	...	

Table is very long showing only small part of it

✓ Explanation:

- FULL OUTER JOIN combines the results of both LEFT and RIGHT JOIN.
- You get **everything** — employees with projects, employees without projects, and even projects with no assigned leads.

 Use this when you want a full picture of both tables.

5. CROSS JOIN — “All possible combinations”

Problem Statement:

Generate all possible combinations of employees and departments.

Query:

```
SELECT
  e.emp_id,
  e.emp_name,
  d.department_name
FROM employees e
CROSS JOIN departments d;
```

emp_id	emp_name	department_name
1	JohnDoe	HR
1	JohnDoe	IT
1	JohnDoe	Marketing
1	JohnDoe	Finance
1	JohnDoe	Operations
2	JaneSmith	HR
2	JaneSmith	IT
2	JaneSmith	Marketing
2	JaneSmith	Finance
2	JaneSmith	Operations
3	BobJohnson	HR
3	BobJohnson	IT
3	BobJohnson	Marketing

Table is very long showing only small part of it

✓ Explanation:

- CROSS JOIN creates a **cartesian product** — every employee with every department.
- If there are 22 employees and 5 departments → you get **110 rows**.
- Useful for *testing, matching, or generating all pair possibilities*.

🎯 *Be careful — it multiplies results fast!*

6. SELF JOIN — “Join a table with itself”

Example Dataset

Let's assume this simplified `employees` table:

emp_id	emp_name	manager_id	
1	John Doe	NULL	
2	Jane Smith	1	
3	Bob Johnson	1	
4	Alice Brown	2	
5	Charlie Davis	2	
6	Eva White	3	
7	Frank Jones	3	
8	Grace Miller	4	

Use Case Example:

You have a table of employees with their `manager_id`.
You want to find each employee's manager name.

Query Example:

```
SELECT
  e.emp_name AS Employee,
  m.emp_name AS Manager
FROM employees e
LEFT JOIN employees m
ON e.manager_id = m.emp_id;
```

✓ This is used when data is hierarchical (like *employee* → *manager*, *product* → *category*, etc.)

Result Table

Employee	Manager
John Doe	NULL
Jane Smith	John Doe
Bob Johnson	John Doe
Alice Brown	Jane Smith
Charlie Davis	Jane Smith
Eva White	Bob Johnson
Frank Jones	Bob Johnson
Grace Miller	Alice Brown

Concept Tip — The Common Column Rule

Always remember this golden rule :

You can only join tables when the **columns being compared are of the same type** and represent the same kind of entity.

For example:

- You can join `emp_department_id` (number) with `department_id` (number).
- But you **cannot** join `emp_name` (text) with `department_id` (number).

. . .

→ Keys & Constraints

Primary Key

What it does: guarantees each row has a unique, non-empty identifier. Use it for the main id you will search by (e.g., `emp_id`, `uid`).

SQL example

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(100)  
);
```

Sample rows

emp_id	emp_name
1	John Doe
2	Jane Smith

What fails: inserting a row with `emp_id = 2` again or inserting with `emp_id = NULL`
→ constraint violation.

Foreign Key

What it does: ensures a value in a *child* table points to an existing row in a *parent* table (keeps references valid; prevents orphans).

SQL example

```
CREATE TABLE departments (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50)  
);  
  
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(100),  
    dept_id INT,  
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)  
);
```

Sample rows

departments

dept_id	dept_name
101	HR
102	IT

employees

emp_id	emp_name	dept_id
1	John Doe	101
2	Jane Smith	102

What fails: inserting `employees(emp_id=3, emp_name='X', dept_id=999)` when `dept_id=999` does not exist → foreign key violation.

CHECK

What it does: enforces a custom rule on values in a column (length, range, pattern). Good for validating input.

SQL example

```
CREATE TABLE customers (
  customer_id INT PRIMARY KEY,
  phone VARCHAR(15) CHECK (LENGTH(phone) = 10)
);
```

Sample rows

customer_id	phone
1	1234567890
2	9876543210

What fails: `INSERT INTO customers VALUES (3, '123')` → fails because phone length ≠ 10.

UNIQUE

What it does: prevents the same value from appearing more than once in the column(s). Useful for emails, usernames, phone numbers. A table may have multiple `UNIQUE` constraints.

SQL example

```
CREATE TABLE users (  
  user_id INT PRIMARY KEY,  
  email VARCHAR(100) UNIQUE  
);
```

Sample rows

user_id	email
101	alice@example.com ↗
102	bob@example.com ↗

What fails: inserting another row with `email = 'alice@example.com'` → duplicate key error.

NOT NULL

What it does: the column must always have a value; it cannot be left empty. Use it for required form fields.

SQL example

```
CREATE TABLE orders (  
  order_id INT PRIMARY KEY,  
  customer_name VARCHAR(100) NOT NULL  
);
```

Sample rows

order_id	customer_name
201	Alice
202	Bob

What fails: `INSERT INTO orders (order_id) VALUES (203)` (no `customer_name`) → fails because `customer_name` is NOT NULL.

DEFAULT

What it does: provides an automatic value when an `INSERT` omits that column. Useful for sensible defaults (status, country, company name).

SQL example

```
CREATE TABLE customers (
  customer_id INT PRIMARY KEY,
  name VARCHAR(50),
  company VARCHAR(50) DEFAULT 'Google'
);
```

Sample rows

customer_id	name	company
301	Alice	Google
302	Bob	Google
303	Carol	Acme

What fails / behavior note: not an error — `DEFAULT` simply supplies the value when omitted. If the column also has `NOT NULL`, the default prevents NULLs when no explicit value is provided.

Quick cheat-sheet (one-line reminders)

- **PRIMARY KEY** → unique + NOT NULL, identifies a row.

- **FOREIGN KEY** → references a primary key in another table (keeps relationships valid).
- **CHECK** → custom rule, e.g. `CHECK (LENGTH(phone)=10)` .
- **UNIQUE** → no two rows can share the same value.
- **NOT NULL** → value must be provided.
- **DEFAULT** → fallback value when omitted.

. . .

SQL Cheatsheet: <https://www.dbvis.com/wp-content/uploads/2024/04/SQL-Cheat-Sheet.pdf>

. . .

If you found this article helpful, don't forget to **share it with your friends** who are also preparing.

*Got doubts or suggestions? Drop them in the **comments** — let's learn and grow together!*

Happy Coding & All the Best for Your Placements! ✨

Sql For Placements

Sql

Sql Analytics

Sql Queries

Data Analyst

Written by Sheel Ganvir

59 followers · 6 following

Software Engineer | Writing some free stuffs which can be valuable!!



[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]